

Programming Basics 1



11: Ausnahmen (Exceptions)

Sommersemester 2025

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

+49(0)40 – 428 125 714

simon.dewert@haw-hamburg.de

Übersicht dieser Lehreinheit

1. Fehlerarten bei der Implementierung
2. Behandlung von Ausnahmen
3. Auslösen von Ausnahmen
4. Eigene Ausnahmen

Fehlerarten bei der Implementierung

Bei der Implementierung können grundsätzlich drei verschiedene Arten von Fehlern unterschieden werden:

Syntaktische Fehler

Verletzung der Syntax der
Programmiersprache

**Detektion durch den
Compiler**

Ausnahmefehler

Verletzung von erst zur Laufzeit
bekannter oder individueller
Umstände und/oder Zustände

Ausnahmebehandlung
(Exception Handling)

Logische Fehler & **Bugs**

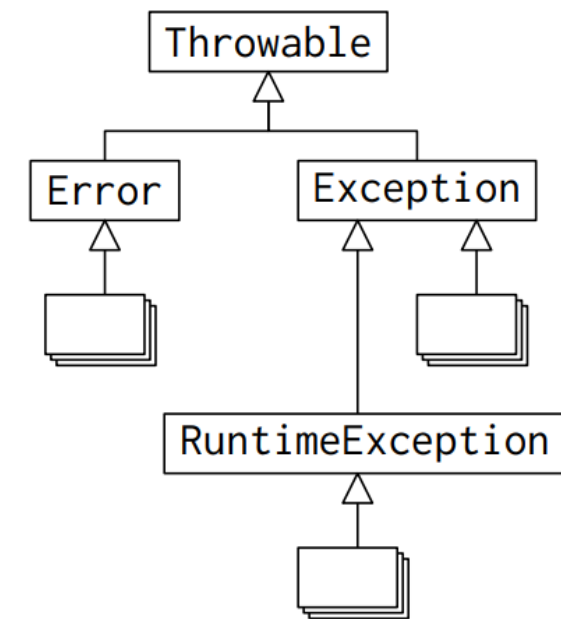
Verletzung der Spezifikation des
Programms

Behebung per Debugging

Laufzeitfehler und Ausnahmen in Java

In Java werden verschiedene Arten von Ausnahmen und Fehlern durch verschiedene Unterklassen von **Throwable** repräsentiert.

- Instanzen von **Error**: **unbehandelbare** unerwartete Fehler, z.B. Fehler der virtuellen Maschine
- Instanzen von **Exception**: vorhersehbare Ausnahmen, welche vom Programm **behandelbar** sind
- Instanzen von **RuntimeException**: Ausnahmen, die **nicht behandelt** werden müssen. **Alle anderen** Instanzen von **Exception** müssen behandelt werden.



Error

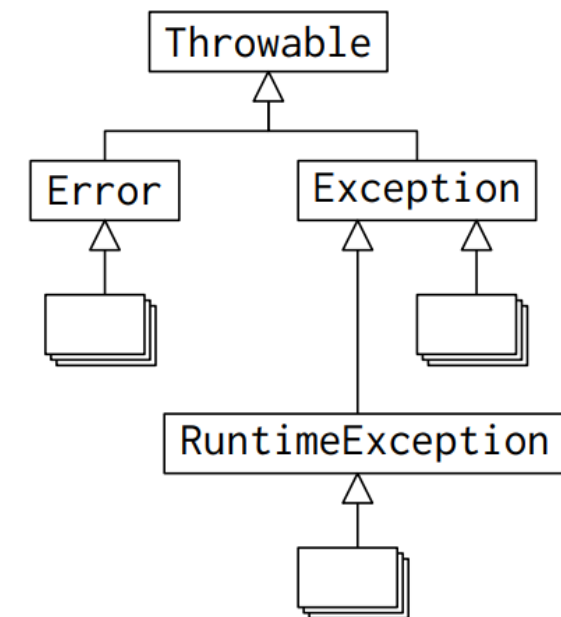


Ein Error tritt in der Regel auf, wenn ein **schwerwiegender Fehler** im Programm oder im System auftritt, der die Ausführung des Programms unmöglich macht oder zu einem instabilen Zustand führt.

Errors sind schwerwiegend und können nicht immer von der laufenden Anwendung erwartet oder behandelt werden.

Ein Beispiel für einen Error ist ein "OutOfMemoryError", der auftritt, wenn einem Programm der verfügbare Arbeitsspeicher ausgeht.

In bestimmten Fällen können bestimmte Fehler, wie z.B. ThreadDeath, als Teil der normalen Programmabwicklung behandelt werden, aber das ist eher die Ausnahme.



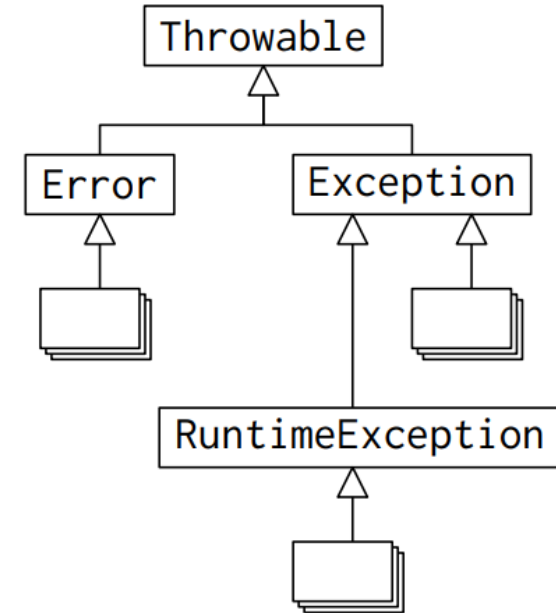
Exception



Eine **Exception** ist ein Ereignis, das während der Ausführung einer Anwendung auftritt und das normale Verhalten des Programms unterbricht.

Exceptions können **erwartet oder unerwartet** sein. Sie können durch Bedingungen wie falsche Benutzereingaben, ungültige Dateizugriffe oder Netzwerkfehler ausgelöst werden. Exceptions können in der Regel abgefangen und behandelt werden, um das Programm zu stabilisieren und spezifische Fehlerbehandlungsmaßnahmen durchzuführen.

Ein Beispiel für eine Exception ist eine "NullPointerException", die auftritt, wenn versucht wird, auf ein Objekt zuzugreifen, das den Wert "null" hat.

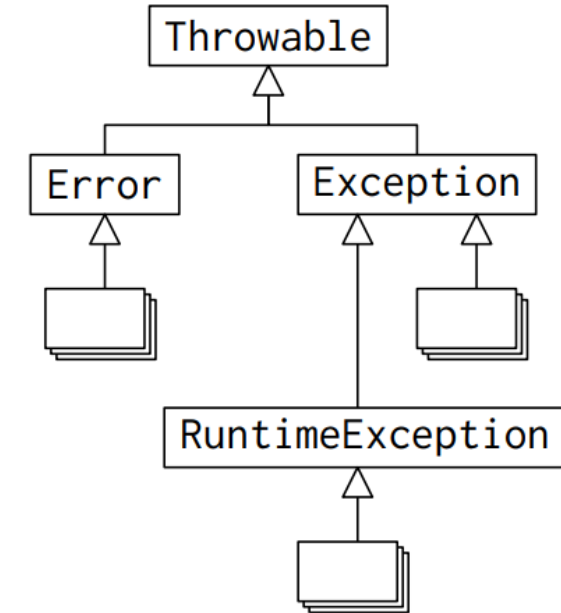


Error vs. Exception



Zusammenfassend lässt sich sagen, dass **Errors** **schwerwiegende** Probleme sind, die oft nicht erwartet oder behandelt werden können, während **Exceptions** Ereignisse darstellen, die während der Ausführung auftreten und normalerweise **erwartet** und **behandelt** werden können.

Die **Behandlung von Exceptions** ermöglicht eine bessere Fehlerbehandlung und **Robustheit von Anwendungen**.



Behandlung einer Ausnahme



- Eine möglicherweise auftretende Ausnahme (**Exception**) kann nur behandelt werden, wenn der sie erzeugende Code innerhalb eines sogenannten **try**-Blocks steht.
- Wenn dies so ist, kann die Ausnahme, die dieser Code eventuell erzeugt (**wirft**), in einem sogenannten **catch**-Block behandelt (**gefangen**) werden:

```
try{  
    [Kritischer Code]  
}  
catch(Exception e) {  
    [Ausnahmebehandlung]  
}
```

} **try**-Block

} **catch**-Block

Aufgabe 11.1

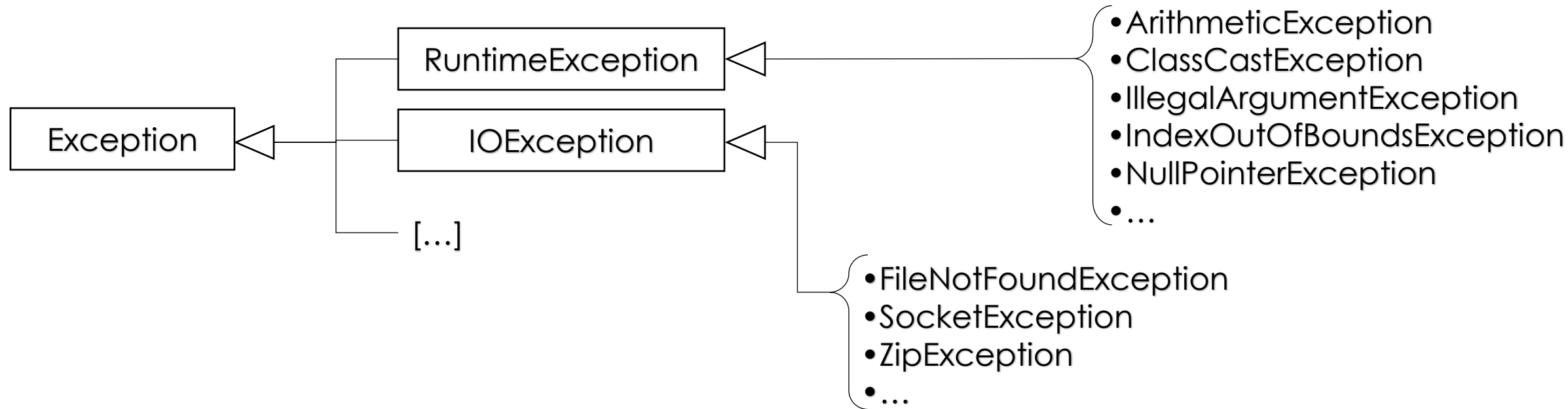


- Legen Sie zunächst ggf. ein neues Java-Projekt namens **vorlesung11** an!
- Erzeugen Sie die Klasse **TryCatch** mit eigener **main()**-Methode.
 - in der **main** sollen zwei Variablen vom Typ **int** erzeugt werden **a = 5;** und **b = 0;**
 - Lassen Sie das Ergebnis von **a / b** in die Konsole ausgeben!
- Lassen das Programm laufen.
 - Behandeln Sie die aufgetretene Ausnahme und starten Sie das Programm erneut. (**try catch**)

Ausnahme-Typen



- Innerhalb eines **try**-Blocks können Instanzen unterschiedlicher Ausnahme-Typen bzw. Exception-Klassen ausgelöst werden.
- Zu den **wichtigsten Exception-Klassen** gehören:



Siehe <https://docs.oracle.com/javase/10/docs/api/java/lang/Exception.html>

Ausnahme-Typen Beispiele



RuntimeException	Beispiel
ArithmeticException	Teilen durch Null
ClassCastException	Klassenumwandlung schlägt fehl
IllegalArgumentException	Argument entspricht nicht den Vorgaben
IndexOutOfBoundsException	Typisch bei Arrays; Zugriff auf einen Index der nicht vorhanden ist.
NullPointerException	Eine leere Referenz soll bearbeitet oder ausgeführt werden.

IOException	Beispiel
FileNotFoundException	Eine Datei kann nicht gefunden/geladen werden.
SocketException	Häufiger Fehler wenn zBsp. Ein Server die Verbindung zum Client verliert
ZipException	Fehler beim öffnen einer Zip-Datei

Ungeprüfte und geprüfte Ausnahmen

Ungeprüften Ausnahmen (sog. "UncheckedExceptions"):

Operationen, die Ausnahmen des Typs **RuntimeException** oder davon abgeleitete Typen auslösen **können** von einem **try**-Block eingefasst sein.

Geprüften Ausnahmen (sog. "CheckedExceptions"):

Operationen, die Ausnahmen **aller anderer Typen** auslösen **müssen** von einem **try**-Block eingefasst sein.

Beispiel:

```
try{
    FileWriter writer = new FileWriter(new File("myFile.txt"));
}
catch (IOException e) {
    e.printStackTrace();
}
```

Konstruktor **KÖNNTE** eine **IOException** werfen; somit **MUSS** er von einem try-Block eingefasst sein! (checked)

Behandlung **mehrerer** Ausnahmen



Innerhalb eines **try**-Blocks kann es durch Ausführung mehrerer Operationen auch zur Auslösung **unterschiedlicher Ausnahmen** kommen. Diese sollten dann jeweils in dedizierten **catch**-Blöcken behandelt werden:

Beispiel:

```
try{
    [Kritischer Code]
}
catch(ArithmeticException e) {
    [...]
}
catch(NullPointerException e) {
    [...]
}
```

catch-Block für den Fall einer
ArithmeticException
(z. B. Divisiondurch 0)

catch-Block für den Fall einer
NullPointerException (z. B. Aufruf einer
Methode mit einer null-Referenz)

Abschluss einer Ausnahmebehandlung

Unabhängig davon, ob eine Operation innerhalb eines **try**-Blocks erfolgreich war oder nicht kann in einem **finally**-Block spezifiziert werden, was **abschließend** passieren soll:

Beispiel:

```
try{
    [Kritischer Code]
}
catch(ArithmeticException e) {
    [...]
}
finally {
    [In jedem Fall auszuführen]
}
```

} **finally**-Block; Anwendungsbeispiel:
Schließen von Dateien und anderen
Ressourcen

- | |
|-------------------------|
| Freund |
| +name: String |
| +sayName(): void |

```
// Beispiel
Freund besterFreund;
Freund[] freunde = new Freund[4];
freunde[0] = new Freund("Siggi");
freunde[1] = new Freund("Peter");
freunde[2] = new Freund("Maggie");
freunde[3] = null;

bestFreund = freunde[3];
bestFreund.sayName();
```

Ausnahme auslösen



- Soll eine Ausnahme innerhalb einer Methode ausgelöst ("geworfen") werden, so ist dies zunächst im Methodenkopf mit dem Schlüsselwort **throws** <Ausnahme-Typ> anzuzeigen.
- Innerhalb der Methode kann die Ausnahme dann mit dem Ausdruck **throw new** <Ausnahme-Konstruktor> ausgelöst bzw. "geworfen" werden:

```
void eineMethode() throws Exception {  
    [...]  
  
    throw new Exception("Fehlermeldung");  
    [...]  
}
```

Hier die Klasse `Exception` als Beispiel;
es könnte auch eine abgeleitete,
speziellere oder eine selbst spezifizierte
Ausnahme sein!

Siehe nächste Folie...

Eigene Ausnahme-Typen spezifizieren

- Eigene Ausnahmen können spezifiziert werden, indem eine **Ableitung** von der Basisklasse `Exception` gebildet wird:

```
class MeineException extends Exception {  
    public MeineException() {  
    }  
  
    public MeineException(String message, <Argumente>) {  
        super(message);  
        [...]  
    }  
}
```

} Optional: Default-Konstruktor

} Optional: Spezifische(r) Konstruktor(en)

Hinweis: Ableitungen von `Exception` sind **geprüfte Ausnahmen** ("CheckedExceptions") und **müssen** mit einem **catch**-Block gefangen werden!

Aufgabe 11.3



- Schreiben Sie ein Programm **PasswortabfrageRestriktiv**, das lediglich die Eingabe eines Passworts fordert. Sollte eines von **drei** von Ihnen definierten, "verbotenen" Passwörtern eingegeben werden, so soll eine selbstdefinierte **IllegalPasswordException** ausgelöst werden:

```
// Beispiel
public class PasswortabfrageRestriktiv
{
    public static void main(String[] args)
    {
        PasswortabfrageRestriktiv pwdSetter = new PasswortabfrageRestriktiv();
        String password = pwdSetter.SetPassword();

        if (!password.equals("")) System.out.println("New password is " + password);
    }
}
```

Beispiel: („Love“, „God“ und „Root“ als verbotene Passwörter):

Password: Root

Illegal password: Root

Eingabe eines „verbotenen“ Passworts

Ausgabe der Exception-Message UND des Passworts

Viele Dank für Ihre Aufmerksamkeit



Übung

Aufgabenstellung:

Sie sollen ein kleines Fahrzeugverwaltungssystem erstellen. Es gibt verschiedene Fahrzeugarten (Auto, Motorrad, LKW), die gemeinsame Eigenschaften haben, sich aber in bestimmten Aspekten unterscheiden. Die gemeinsame Basis bildet eine abstrakte Klasse Fahrzeug.

1. Abstrakte Klasse Fahrzeug
Erstellen Sie eine abstrakte Klasse Fahrzeug mit den folgenden geschützten Attributen:
protected String hersteller;
protected String modell;
protected int baujahr;
protected double kilometerstand;

Fügen Sie hinzu:
 - Konstruktor zur Initialisierung aller Attribute
 - Getter und Setter
 - Eine abstrakte Methode void starten()
 - Eine abstrakte Methode void infoAusgeben()
 - Eine überschriebene toString()-Methode mit sinnvoller Ausgabe
2. Klasse Auto
Erbt von Fahrzeug. Zusätzliche Eigenschaften:
private int anzahlTueren;
private boolean istElektro;
Implementiert starten(): „Das Auto [Modell] startet per [Zündschlüssel/Strom].“
Implementiert infoAusgeben(): Gibt alle relevanten Daten auf der Konsole aus.
toString() erweitert um Autodetails.
3. Klasse Motorrad
Erbt von Fahrzeug. Zusätzliche Eigenschaft:
private boolean hatBeiwagen;
starten(): „Motorrad [Modell] wird mit Kickstart gestartet.“
infoAusgeben(): Gibt alle Daten aus. toString() erweitert.
4. Klasse Lkw
Erbt von Fahrzeug. Zusätzliche Eigenschaften:
private double ladegewicht;
private int achsen; starten(): „LKW [Modell] mit [Achsen] Achsen startet den Motor.“
infoAusgeben() zeigt alle Daten. toString() erweitert.
5. Testklasse FahrzeugTest
Erstellen Sie eine main()-Methode, die Folgendes tut:
 - Erzeugt mehrere Fahrzeuge unterschiedlichen Typs.
 - Speichert sie in einem ArrayList<Fahrzeug> oder Fahrzeug[].
 - Iteriert durch alle Objekte:
Ruft starten() auf
Gibt infoAusgeben() aus
 - Optionale Erweiterungen (für Vertiefung):
Methode void fahren(double kilometer) in Fahrzeug, die den Kilometerstand erhöht.
Abstrakte Methode getFahrzeugTyp(): Gibt den Typ als String zurück.
Statische Methode zur Zählung, wie viele Fahrzeuge insgesamt erstellt wurden.
Eine FahrzeugFactory-Klasse zur objektorientierten Erstellung von Fahrzeugen (optional mit Scanner für Konsoleneingabe).